Day 01: Full Stack Development Fundamentals

1. What is Full Stack Development?

Full Stack Development is the process of developing both the frontend and backend of a web application, including database integration.

It consists of three main components:

Frontend: The user interface of the application.

Backend: Handles business logic and processes requests.

Database: Stores and manages application data.

Example: In an online shopping application, the frontend displays products, the backend processes orders, and the database stores product and customer information.

2. What is the difference between Frontend, Backend, and Database?

Frontend: The frontend is the part of an application that users see and interact with.

Technologies: HTML, CSS, JavaScript, React, and Angular.

Backend: The backend processes requests, implements business logic, and communicates with the database.

Technologies: Java, Spring Boot, and REST APIs.

Database: The database stores, retrieves, and manages application data.

Technologies: MySQL and PostgreSQL.

Example: When a user logs in, the frontend collects the credentials, the backend validates them, and the database provides the stored user information needed for verification.

3. Explain Client–Server Architecture.

Client–Server Architecture is a model in which a client sends requests to a server, and the server processes those requests and returns responses.

Client: Usually a web browser or mobile application.

Server: Processes requests and applies business logic.

Database: Stores and retrieves application data.

Example: When a user opens an online shopping website, the browser requests product information from the server. The server retrieves the required information from the database and returns it to the browser.

Architecture flow:

Client (Frontend) → Server (Backend) → Database

The response travels back to the client through the server.

4. Explain the HTTP Request–Response Flow.

HTTP stands for Hypertext Transfer Protocol. It enables communication between a client and a web server.

The request–response flow works as follows:

The user performs an action on the frontend.

The frontend sends an HTTP request to the backend.

The backend receives and validates the request.

The backend communicates with the database if required.

The backend sends an HTTP response to the client.

The frontend processes the response and displays the result.

Example: When a user searches for a product, the frontend sends a GET request. The backend retrieves matching products from the database and returns the results, usually in JSON format.

Common HTTP methods:

GET: Retrieves data.

POST: Creates a resource or submits data.

PUT: Replaces or updates a resource.

PATCH: Updates part of a resource.

DELETE: Deletes a resource.

5. What is a REST API?

API stands for Application Programming Interface. It allows different software components to communicate with each other.

REST stands for Representational State Transfer. A REST API is an interface that commonly uses HTTP methods to access and manipulate resources.

Example endpoint: GET /api/products

This endpoint retrieves product information from the backend.

A REST API typically:

Uses HTTP methods such as GET, POST, PUT, and DELETE.

Identifies resources using URLs.

Exchanges data using formats such as JSON.

Follows stateless communication, meaning each request contains the information needed to process it.

Example JSON response:

```
{
  "id": 1,
  "name": "Laptop",
  "price": 50000
}
```

Here, the backend returns product information to the frontend through the API.

6. What Is the Java Full Stack Technology Stack?

A Java Full Stack Developer uses frontend, backend, and database technologies to develop complete web applications.

Component

Technologies

Purpose

Frontend

HTML, CSS, JavaScript

Builds the user interface

Frontend Framework

React or Angular

Develops interactive interfaces

Programming Language

Java

Implements backend logic

Backend Framework

Spring Boot

Develops backend applications and REST APIs

Database

MySQL

Stores application data

Persistence

Hibernate, Spring Data JPA

Connects Java applications with databases

API Testing

Postman

Tests REST APIs

Unit Testing

JUnit, Mockito

Tests application code

Version Control

Git, GitHub

Tracks and manages code changes

Build Tool

Maven or Gradle

Manages dependencies and builds applications

Example technology stack:

HTML + CSS + JavaScript + React + Java + Spring Boot + Hibernate + MySQL.

These technologies work together to build, test, and deploy a web application.

7. Explain a Real-World Application Using Full Stack Architecture.

Example: Online Shopping Application

An online shopping application allows users to browse products, add items to a cart, and place orders.

Step 1: Frontend

The frontend is developed using HTML, CSS, JavaScript, and React. It displays products, search options, and the shopping cart.

Step 2: Backend

The backend is developed using Java and Spring Boot. It processes user requests, validates information, manages orders, and implements business logic.

Step 3: REST API

The frontend communicates with the backend through REST APIs.

Examples:

GET /api/products — retrieves products.

POST /api/orders — creates an order.

GET /api/orders/101 — retrieves order information.

Step 4: Database

MySQL stores product details, user information, cart data, and order records.

Step 5: Request and Response

When a user searches for a product, the frontend sends a request to the backend. The backend retrieves matching products from MySQL and returns the results as JSON. The frontend then displays the products.

Architecture:

USER

|

v

FRONTEND (React)

|

| HTTP Request

v

REST API (Spring Boot)

|

v

BUSINESS LOGIC

|

v

DATABASE (MySQL)

|

v

Query Result

|

v

BACKEND RESPONSE

|

v

FRONTEND DISPLAYS DATA

Conclusion: Full Stack Development combines frontend development, backend programming, API integration, and database management to create complete web applications. Java Full Stack Development commonly uses Java, Spring Boot, REST APIs, and MySQL for the backend and database, along with modern frontend technologies.